

# Reskilling & Job-Hunt Roadmap (9+ YOE)

Azure → Docker → Kubernetes → Terraform → Node.js → React → Next.js → Traditional AI/ML → Vector DBs → Gen AI/RAG

**Rule:** One topic at a time, in order. Do not start topic 2 until topic 1’s checklist is done. Within each topic, learn sub-topics in the sequence listed (earlier items are prerequisites for later ones).

## CONTENTS

- Part 0 — Immediate Actions

Part 2 — Detailed Topic Breakdowns

1. Azure Fundamentals

3. Kubernetes

5. Node.js

7. Next.js

9. Vector Databases

Quick Reference — Cert Order
- Part 1 — High-Level Sequence Checklist

Part 3 — Senior-Level Extras

2. Docker

4. Terraform

6. React

8. Traditional AI / Machine Learning

10. Gen AI / RAG

## PART 0 — IMMEDIATE ACTIONS

Start today. Run throughout the entire roadmap.

- ☐ Update resume with quantified achievements
- ☐ Update LinkedIn headline + “Open to Work” (recruiters only, if needed)
- ☐ Message 10 people in your network that you’re looking
- ☐ Apply to 5+ roles this week
- ☐ Set a recurring 45-min slot, 2–3×/week, for system design + behavioral practice (starts Week 2, runs to the end)
- ☐ Keep applying every single week from here on — don’t pause for certs

## PART 1 — HIGH-LEVEL SEQUENCE CHECKLIST

| WEEK | TOPIC              | DAILY PACE    | CHECKLIST                                                                                                                                                  |
|------|--------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1–3  | Azure Fundamentals | 1–1.5 hrs/day | <div><input type="checkbox"/> Concepts done</div> <div><input type="checkbox"/> AZ-900 exam booked</div> <div><input type="checkbox"/> AZ-900 passed</div> |
| 4–6  | Docker             | 1–1.5 hrs/day | <div><input type="checkbox"/> Concepts + hands-on done</div> <div><input type="checkbox"/> Can containerize an app</div>                                   |
| 7–9  | Kubernetes         | 1.5 hrs/day   | <div><input type="checkbox"/> Concepts + hands-on done</div> <div><input type="checkbox"/> Can deploy an app to a local cluster (minikube/kind)</div>      |

| WEEK      | TOPIC                      | DAILY PACE        | CHECKLIST                                                                                                                                                                                              |
|-----------|----------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10–12     | Terraform                  | 1.5 hrs/day       | <input type="checkbox"/> Concepts + hands-on done<br><input type="checkbox"/> Terraform Associate exam booked<br><input type="checkbox"/> Terraform Associate 004 passed                               |
| 13–15     | Node.js                    | 1–1.5 hrs/day     | <input type="checkbox"/> Concepts done<br><input type="checkbox"/> One API built end to end                                                                                                            |
| 16–17     | React                      | 1–1.5 hrs/day     | <input type="checkbox"/> Concepts done<br><input type="checkbox"/> One app built end to end                                                                                                            |
| 18        | Next.js                    | 1.5 hrs/day       | <input type="checkbox"/> Concepts done<br><input type="checkbox"/> App migrated/rebuilt in Next.js                                                                                                     |
| 19–20     | Traditional AI/ML          | 1–1.5 hrs/day     | <input type="checkbox"/> Concepts done (fluency only — no coding required)                                                                                                                             |
| 21        | Vector Databases           | 1 hr/day          | <input type="checkbox"/> Concepts done<br><input type="checkbox"/> One small embedding search demo built                                                                                               |
| 22–23     | Gen AI / RAG               | 1.5–2 hrs/day     | <input type="checkbox"/> Concepts done<br><input type="checkbox"/> Mini RAG chatbot built<br><input type="checkbox"/> AI-901 exam booked<br><input type="checkbox"/> AI-901 passed                     |
| 24–25     | Integration Project        | 2 hrs/day         | <input type="checkbox"/> Next.js + Node API + RAG chatbot built<br><input type="checkbox"/> Containerized with Docker<br><input type="checkbox"/> Deployed to Azure (AKS or App Service) via Terraform |
| From Wk 2 | System Design + Behavioral | 45 min, 2–3×/week | <input type="checkbox"/> 2 system designs/week practiced<br><input type="checkbox"/> 6+ STAR stories ready                                                                                             |

~25 weeks (~6 months) at a steady pace. If you already have strong exposure in one area, compress that section and bank the time for weaker areas — don't skip the sequence, just move faster through what you know.

**Part 2 — Detailed Topic Breakdowns.** Each topic has: Architecture → Sequenced sub-topics checklist → 30 interview questions (priority order).

# 1. AZURE FUNDAMENTALS

## ARCHITECTURE — HOW IT FITS TOGETHER

Azure is organized as:



All of this is deployed into Regions made up of Availability Zones for resilience.

Identity flows through Entra ID (Azure AD), which issues tokens that RBAC checks against resources. Networking (VNets/subnets) isolates resources, and Azure Policy + Cost Management enforce governance across all of it.

Everything you build later (Docker containers, Kubernetes clusters, Terraform-provisioned infra) ultimately lives inside this hierarchy.

## SEQUENCED CHECKLIST

- ☐ 1. Core architectural components: regions, availability zones, resource groups, subscriptions, management groups
- ☐ 2. Identity: Entra ID (Azure AD), users/groups, RBAC roles, managed identities
- ☐ 3. Networking basics: VNets, subnets, NSGs, load balancers, DNS
- ☐ 4. Compute: VMs, App Service, Azure Functions, AKS (concept only for now)
- ☐ 5. Storage: Blob/Disk/File, redundancy (LRS/ZRS/GRS)
- ☐ 6. Databases: Azure SQL, Cosmos DB (concept level)
- ☐ 7. Governance: Azure Policy, tags, cost management/budgets
- ☐ 8. Monitoring: Azure Monitor, Log Analytics, alerts
- ☐ 9. Azure AI services overview (Cognitive Services, Azure OpenAI, AI Foundry)
- ☐ 10. Shared responsibility model + compliance basics
- ☐ Book and pass AZ-900

## TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15 |                                                                                                    | QUESTIONS 16-30 |                                                                               |
|----------------|----------------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------|
| 1.             | What is the relationship between management groups, subscriptions, resource groups, and resources? | 16.             | What is Azure Policy used for, and how does it differ from RBAC?              |
| 2.             | Explain the difference between a region and an availability zone.                                  | 17.             | Explain how DNS resolution works within a VNet.                               |
| 3.             | What's the difference between Azure AD (Entra ID) RBAC and Azure Policy?                           | 18.             | What's a service principal, and how does it differ from a managed identity?   |
| 4.             | What is a managed identity and why is it preferred over storing credentials in code?               | 19.             | How would you securely store secrets for an application running in Azure?     |
| 5.             | Explain LRS vs ZRS vs GRS storage redundancy — trade-offs?                                         | 20.             | What's the difference between horizontal and vertical scaling in App Service? |
| 6.             | What's the difference between Azure Functions and App Service?                                     | 21.             | Explain Azure's disaster recovery options (backup, geo-redundancy, failover). |
| 7.             | When would you choose a VM over App Service or Functions?                                          | 22.             | What's an ARM template, and how does it relate to Terraform?                  |
| 8.             | How does an NSG (Network Security Group) work?                                                     | 23.             | How do you peer two VNets, and why would you need to?                         |
| 9.             | What's the difference between a public and private VNet subnet?                                    | 24.             | What's the difference between Standard and Premium storage tiers?             |
| 10.            | How would you design a highly available architecture across regions?                               | 25.             | How does Azure handle autoscaling for VMs vs App Service?                     |
| 11.            | Explain the shared responsibility model in cloud computing.                                        | 26.             | What's Azure AI Foundry, and how does it relate to Azure OpenAI Service?      |
| 12.            | How do you control cloud costs at scale (tagging, budgets, reserved instances)?                    | 27.             | Explain how you'd set up least-privilege access for a team in Azure.          |
| 13.            | What monitoring/alerting would you set up for a production app?                                    | 28.             | What's the difference between an NSG and Azure Firewall?                      |
| 14.            | What's the difference between Azure SQL and Cosmos DB?                                             | 29.             | How would you migrate an on-prem application to Azure with minimal downtime?  |
| 15.            | How does Azure Load Balancer differ from Application Gateway?                                      | 30.             | Walk me through designing a 3-tier web app architecture in Azure end to end.  |

## 2. DOCKER

### ARCHITECTURE — HOW IT FITS TOGETHER

Docker packages an application into:

A Dockerfile (the build instructions) → An Image (a read-only template, built in layers) → A Container (a running instance of that image, isolated using the host OS kernel — not a full VM)

The Docker Engine (daemon) manages images, containers, networks, and volumes on a host. Volumes persist data outside the container's writable layer. Docker Compose lets you define and run multi-container apps (e.g., app + database) together.

This containerization is the foundation Kubernetes later orchestrates at scale.

### SEQUENCED CHECKLIST

- ☐ 1. What a container is vs a VM (shared kernel vs full OS)
- ☐ 2. Images vs containers — image layers, caching
- ☐ 3. Writing a Dockerfile (FROM, RUN, COPY, CMD, ENTRYPOINT, EXPOSE)
- ☐ 4. Building and tagging images (docker build, docker tag)
- ☐ 5. Running/managing containers (run, ps, exec, logs, stop, rm)
- ☐ 6. Volumes and bind mounts — persisting data
- ☐ 7. Networking: bridge networks, port mapping, container-to-container communication
- ☐ 8. Multi-stage builds (reducing image size)
- ☐ 9. Docker Compose — multi-container apps
- ☐ 10. Image registries (Docker Hub, Azure Container Registry) — push/pull
- ☐ 11. Basic security: minimizing image size/attack surface, non-root users

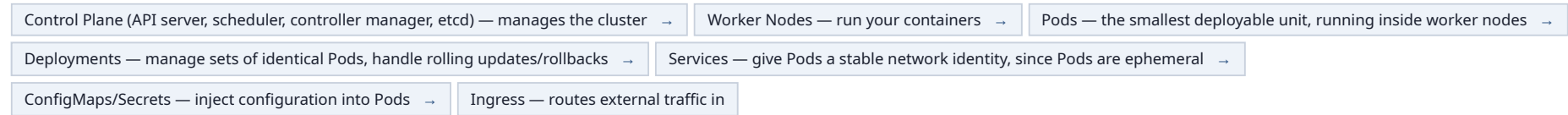
### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                                     | QUESTIONS 16-30                                                                             |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 1. What's the difference between a container and a virtual machine?                | 16. Explain the difference between COPY and ADD in a Dockerfile.                            |
| 2. Explain the difference between a Docker image and a Docker container.           | 17. How do you limit CPU/memory resources for a container?                                  |
| 3. What are image layers, and why do they matter for build speed and size?         | 18. What happens to a container's filesystem changes when it stops vs is removed?           |
| 4. Walk me through a Dockerfile you've written — explain each instruction.         | 19. How would you run a container as a non-root user, and why does it matter?               |
| 5. What's the difference between CMD and ENTRYPOINT?                               | 20. What's the difference between an ENTRYPOINT script and just running the app directly?   |
| 6. How do you persist data in a container that would otherwise be lost on restart? | 21. How do health checks work in Docker?                                                    |
| 7. What's the difference between a volume and a bind mount?                        | 22. What's .dockerignore used for?                                                          |
| 8. How does container networking work — bridge vs host vs none?                    | 23. How do you handle logging for containerized applications?                               |
| 9. What is a multi-stage build, and why would you use one?                         | 24. What's the difference between docker stop and docker kill?                              |
| 10. How do you reduce the size of a Docker image?                                  | 25. How would you containerize a Node.js app vs a Python app?                               |
| 11. What's the difference between docker run and docker exec?                      | 26. What are the security risks of running containers, and how do you mitigate them?        |
| 12. How would you debug a container that keeps crashing/restarting?                | 27. How do you version and tag images for a CI/CD pipeline?                                 |
| 13. What's Docker Compose used for?                                                | 28. What's the difference between an anonymous volume and a named volume?                   |
| 14. How do you pass environment variables/secrets into a container securely?       | 29. How does Docker networking differ when containers are on the same host vs across hosts? |
| 15. What's a container registry, and how do you push/pull images from one?         | 30. Describe how you'd migrate a legacy monolith into a containerized setup.                |

### 3. KUBERNETES

#### ARCHITECTURE — HOW IT FITS TOGETHER

Kubernetes orchestrates containers (built with Docker) across a cluster of machines, structured as:



This is the layer that takes what Docker built and runs it reliably, at scale, self-healing — and it's what Terraform will later provision (e.g., an AKS cluster).

#### SEQUENCED CHECKLIST

- ☐ 1. Why orchestration is needed (beyond single-host Docker)
- ☐ 2. Cluster architecture: control plane vs worker nodes
- ☐ 3. Pods — the smallest deployable unit
- ☐ 4. Deployments — replica sets, rolling updates, rollbacks
- ☐ 5. Services — ClusterIP, NodePort, LoadBalancer
- ☐ 6. ConfigMaps and Secrets
- ☐ 7. Ingress — routing external traffic
- ☐ 8. Namespaces — isolating environments/teams
- ☐ 9. Persistent Volumes / Persistent Volume Claims
- ☐ 10. Horizontal Pod Autoscaler
- ☐ 11. Health checks: liveness vs readiness probes
- ☐ 12. Helm (concept level) — packaging Kubernetes manifests
- ☐ 13. Hands-on: deploy a sample app to minikube or kind locally

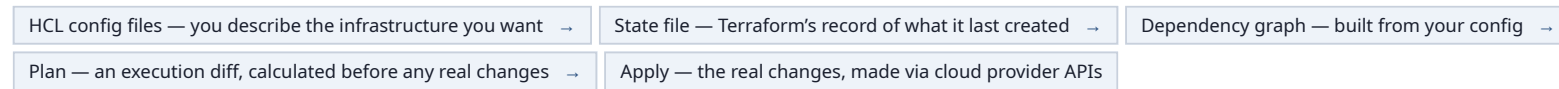
#### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                                                             | QUESTIONS 16-30                                                                            |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1. What problem does Kubernetes solve that plain Docker doesn't?                                           | 16. What is a DaemonSet, and when would you use one?                                       |
| 2. Explain the Kubernetes control plane components and their roles.                                        | 17. How does Kubernetes service discovery work internally (DNS)?                           |
| 3. What is a Pod, and why isn't it usually just one container forever guaranteed?                          | 18. What's a Job vs a CronJob in Kubernetes?                                               |
| 4. What's the difference between a Deployment and a ReplicaSet?                                            | 19. How would you debug a Pod stuck in CrashLoopBackOff?                                   |
| 5. How does a rolling update work, and how do you roll one back?                                           | 20. What's the difference between kubectl apply and kubectl create?                        |
| 6. Why do Pods need a Service, and what are the differences between ClusterIP, NodePort, and LoadBalancer? | 21. How do resource requests and limits work?                                              |
| 7. What's the difference between a ConfigMap and a Secret?                                                 | 22. What is Helm, and how does it simplify Kubernetes deployments?                         |
| 8. Explain liveness vs readiness probes.                                                                   | 23. How do you manage secrets securely in Kubernetes (vs plain Secrets objects)?           |
| 9. What is Ingress, and how does it differ from a LoadBalancer Service?                                    | 24. What's a sidecar container pattern?                                                    |
| 10. How does the Horizontal Pod Autoscaler decide when to scale?                                           | 25. How does Kubernetes networking allow Pod-to-Pod communication across nodes?            |
| 11. What's a Namespace used for?                                                                           | 26. What's the difference between AKS-managed control plane and a self-managed cluster?    |
| 12. Explain Persistent Volumes vs Persistent Volume Claims.                                                | 27. How would you set up multi-environment isolation in one cluster?                       |
| 13. How does Kubernetes handle a node failure?                                                             | 28. What's a taint/toleration, and what problem does it solve?                             |
| 14. What's the difference between a StatefulSet and a Deployment?                                          | 29. How do you monitor and log a Kubernetes cluster in production?                         |
| 15. How do you perform a zero-downtime deployment in Kubernetes?                                           | 30. Walk me through designing a Kubernetes deployment strategy for a high-traffic web app. |

## 4. TERRAFORM

### ARCHITECTURE — HOW IT FITS TOGETHER

Terraform is Infrastructure as Code. The workflow flows as:



Providers are the plugins that talk to a specific platform (e.g., AzureRM). Modules package reusable chunks of config. Remote state lets teams collaborate safely with locking, so two people can't apply conflicting changes at once.

This is the layer that actually provisions the Azure infrastructure and Kubernetes clusters you learned about above.

### SEQUENCED CHECKLIST

- ☐ 1. Why Infrastructure as Code matters
- ☐ 2. Core workflow: init, plan, apply, destroy
- ☐ 3. Providers, resources, data sources
- ☐ 4. Variables, outputs, locals
- ☐ 5. State — what it is, remote state + locking
- ☐ 6. terraform state mv/rm/import/list/show
- ☐ 7. Modules — writing reusable ones, versioning
- ☐ 8. for\_each vs count
- ☐ 9. Workspaces vs separate state files for environments
- ☐ 10. Lifecycle rules (create\_before\_destroy, prevent\_destroy, ignore\_changes)
- ☐ 11. Dependency graph — implicit vs explicit (depends\_on)
- ☐ 12. Secrets handling (ephemeral values, write-only arguments)
- ☐ 13. HCP Terraform: remote runs, policy checks (Sentinel/OPA)
- ☐ 14. Hands-on: provision a VM/App Service + an AKS cluster with Terraform
- ☐ Book and pass Terraform Associate 004

### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1–15                                                                       | QUESTIONS 16–30                                                             |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1. What problem does Infrastructure as Code solve?                                   | 16. How do you version and pin providers/modules?                           |
| 2. What is Terraform state, and why is it dangerous to edit manually?                | 17. What's terraform refresh and when would you need it?                    |
| 3. Walk through the init → plan → apply workflow.                                    | 18. Explain create_before_destroy and give a scenario.                      |
| 4. How do you manage state for multiple environments?                                | 19. What's the difference between local and remote state?                   |
| 5. Explain for_each vs count.                                                        | 20. How would you roll back a bad apply?                                    |
| 6. How do you import an existing cloud resource into Terraform management?           | 21. What makes a Terraform module reusable vs project-specific?             |
| 7. What happens if two people run apply at the same time, and how do you prevent it? | 22. How do you handle Terraform in a CI/CD pipeline safely?                 |
| 8. How do you structure a large Terraform codebase?                                  | 23. What's HCP Terraform, and what does it add over open-source Terraform?  |
| 9. What's the difference between a provider and a provisioner?                       | 24. How do policy-as-code tools fit into a Terraform workflow?              |
| 10. How would you handle a secret in Terraform without exposing it in state or logs? | 25. What's the purpose of the .terraform.lock.hcl file?                     |
| 11. Describe a time you dealt with configuration drift.                              | 26. How do you handle a resource that must be destroyed/recreated?          |
| 12. What's your strategy for zero-downtime infrastructure changes?                   | 27. What are ephemeral values / write-only arguments?                       |
| 13. What's the difference between a resource and a data source?                      | 28. How would you provision an AKS cluster with Terraform, at a high level? |
| 14. How does Terraform build its dependency graph?                                   | 29. What's the blast radius concern with a single large state file?         |
| 15. What's the difference between Terraform workspaces and separate state files?     | 30. Walk me through Terraforming an entire environment from scratch.        |

## 5. NODE.JS

### ARCHITECTURE — HOW IT FITS TOGETHER

Node.js runs JavaScript outside the browser using the V8 engine, built around a single-threaded event loop with non-blocking I/O. Expensive operations are handed off so the main thread stays free, and a callback/Promise fires when work completes.

On top of this, Express adds a middleware chain that processes each incoming request as:



This is the backend layer that exposes your APIs (including the RAG chatbot's backend) and gets containerized with Docker, deployed via Kubernetes/Terraform above.

### SEQUENCED CHECKLIST

- |                                                                                        |                                                                                       |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <input type="checkbox"/> 1. Event loop and non-blocking I/O                            | <input type="checkbox"/> 2. Callbacks → Promises → async/await                        |
| <input type="checkbox"/> 3. CommonJS (require) vs ES Modules (import)                  | <input type="checkbox"/> 4. Error handling patterns                                   |
| <input type="checkbox"/> 5. Express: routing, middleware chain, request/response cycle | <input type="checkbox"/> 6. Building a REST API: status codes, versioning, validation |
| <input type="checkbox"/> 7. Authentication basics: sessions vs JWT vs OAuth            | <input type="checkbox"/> 8. Streams and buffers                                       |
| <input type="checkbox"/> 9. Environment config/secrets management                      | <input type="checkbox"/> 10. Testing basics (Jest/Mocha)                              |
| <input type="checkbox"/> 11. Scaling: clustering, worker threads, memory leaks         | <input type="checkbox"/> Hands-on: build one small REST API end to end                |

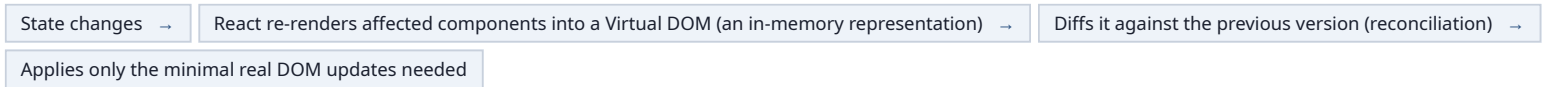
### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                                       | QUESTIONS 16-30                                                                                 |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1. Explain the Node.js event loop.                                                   | 16. How do you manage environment-specific secrets in Node?                                     |
| 2. What's the difference between process.nextTick, setImmediate, and setTimeout?     | 17. What's the Node.js cluster module vs worker threads?                                        |
| 3. How do you handle errors in async/await code?                                     | 18. How would you version a REST API without breaking clients?                                  |
| 4. What's the difference between CommonJS and ES Modules?                            | 19. What's CORS, and how do you configure it correctly?                                         |
| 5. What's middleware in Express?                                                     | 20. How do you test an Express route that depends on a database?                                |
| 6. How do you secure a REST API?                                                     | 21. What's the difference between npm and npx?                                                  |
| 7. Explain JWT-based auth vs session-based auth.                                     | 22. How do you handle graceful shutdown of a Node server?                                       |
| 8. How would you scale a Node.js app across multiple CPU cores?                      | 23. What's event-driven architecture, and how does EventEmitter relate?                         |
| 9. When would you use a stream instead of loading a whole file into memory?          | 24. How would you implement pagination for a large dataset?                                     |
| 10. How do you debug a memory leak in a Node app?                                    | 25. Sync vs async validation in a request pipeline?                                             |
| 11. What's the difference between a Promise and an async function under the hood?    | 26. How do you prevent injection attacks in a Node app?                                         |
| 12. How do you handle unhandled promise rejections?                                  | 27. What's the purpose of a .env file, and its risks?                                           |
| 13. What's the purpose of centralized error-handling middleware?                     | 28. How would you structure a Node.js project for a mid-size team?                              |
| 14. How would you design rate limiting for a public API?                             | 29. What's backpressure in streams?                                                             |
| 15. What's the difference between readable, writable, duplex, and transform streams? | 30. Walk me through designing a Node.js API handling uploads, auth, and rate limiting together. |

6. REACT

ARCHITECTURE — HOW IT FITS TOGETHER

React builds UIs as a tree of components describing what the UI should look like given current state and props. When state changes, the render flow is:



Hooks let function components hold state and side effects without classes. Data typically flows down via props; when many components need shared state, Context or a state library avoids “prop drilling.” This component tree is what Next.js wraps with routing, rendering strategy, and server/client boundaries.

SEQUENCED CHECKLIST

- ☐ 1. Components, props, and state — the basics
- ☐ 2. JSX and rendering
- ☐ 3. Core hooks: useState, useEffect
- ☐ 4. Virtual DOM and reconciliation
- ☐ 5. Lists and keys
- ☐ 6. Controlled vs uncontrolled components
- ☐ 7. Prop drilling → Context API
- ☐ 8. Performance hooks: useMemo, useCallback, React.memo
- ☐ 9. Custom hooks
- ☐ 10. Error boundaries
- ☐ 11. State management libraries (Redux/Zustand)
- ☐ 12. Testing basics (React Testing Library)
- ☐ Hands-on: build one small app

TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                       | QUESTIONS 16-30                                                 |
|----------------------------------------------------------------------|-----------------------------------------------------------------|
| 1. Explain the Virtual DOM and reconciliation.                       | 16. What's a render prop pattern?                               |
| 2. What's the difference between state and props?                    | 17. How would you debug a component re-rendering too often?     |
| 3. useMemo vs useCallback — when do you actually need them?          | 18. Sync vs async state updates in React 18+?                   |
| 4. Why does React need a key prop in lists?                          | 19. Data fetching pitfalls: race conditions, cleanup?           |
| 5. Context API vs Redux/Zustand — when to use which?                 | 20. What's code-splitting (React.lazy, Suspense)?               |
| 6. What's a custom hook you've built?                                | 21. Reconciliation when component types change between renders? |
| 7. How do error boundaries work, and what don't they catch?          | 22. useRef vs useState?                                         |
| 8. Controlled vs uncontrolled components — trade-offs?               | 23. Sharing state between unrelated sibling components?         |
| 9. useEffect dependency array behaviors — empty vs none vs specific? | 24. Common cause of memory leaks in React components?           |
| 10. How do you avoid unnecessary re-renders in a large tree?         | 25. Testing a component that depends on an API call?            |
| 11. Class lifecycle methods vs hooks?                                | 26. Shallow vs deep comparison in React?                        |
| 12. What is “lifting state up”?                                      | 27. Implementing a debounced search input?                      |
| 13. How does React batch state updates?                              | 28. Purpose of StrictMode?                                      |
| 14. React.memo vs useMemo?                                           | 29. Global error handling across a React app?                   |
| 15. How do you handle forms with many fields efficiently?            | 30. Walk me through architecting a medium-sized React app.      |



## 7. NEXT.JS

### ARCHITECTURE — HOW IT FITS TOGETHER

Next.js is a framework built on React that adds routing, multiple rendering strategies, and a server layer. Rendering strategy options:

SSR — fresh HTML rendered on every request →

SSG — built once at build time →

ISR — built once, then revalidated on a schedule/on-demand

With the App Router, components are Server Components by default (rendered on the server, zero client JS shipped) unless explicitly marked “use client”. Route Handlers let you build API endpoints inside the same app.

This is the framework layer your RAG chatbot’s frontend and lightweight backend logic will likely live in.

### SEQUENCED CHECKLIST

- |                                                                      |                                                                                        |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <input type="checkbox"/> 1. App Router vs Pages Router               | <input type="checkbox"/> 2. File-based routing basics                                  |
| <input type="checkbox"/> 3. Server Components vs Client Components   | <input type="checkbox"/> 4. Rendering strategies: SSR vs SSG vs ISR vs CSR             |
| <input type="checkbox"/> 5. Data fetching patterns and caching       | <input type="checkbox"/> 6. Route Handlers (API routes)                                |
| <input type="checkbox"/> 7. Middleware                               | <input type="checkbox"/> 8. Image and font optimization                                |
| <input type="checkbox"/> 9. Auth patterns in Next.js                 | <input type="checkbox"/> 10. Deployment considerations (Vercel vs Azure/containerized) |
| <input type="checkbox"/> Hands-on: rebuild your React app in Next.js |                                                                                        |

### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                 | QUESTIONS 16-30                                                                         |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1. SSR vs SSG vs ISR vs CSR — how do you choose?               | 16. Implementing ISR for hourly-updating pages?                                         |
| 2. Server Component vs Client Component?                       | 17. fetch caching in Next.js vs plain fetch?                                            |
| 3. Why Server Components by default, when to add “use client”? | 18. Server-only vs NEXT_PUBLIC_ env vars?                                               |
| 4. How does Next.js caching work, how do you invalidate it?    | 19. Containerizing a Next.js app with Docker?                                           |
| 5. Structuring Route Handlers for backend-for-frontend?        | 20. What’s a Route Group, why use one?                                                  |
| 6. Auth: server-side vs client-side in Next.js?                | 21. SEO handling (metadata API, sitemaps)?                                              |
| 7. App Router vs Pages Router?                                 | 22. Static params generation vs dynamic rendering?                                      |
| 8. File-based routing, dynamic routes ([id])?                  | 23. Protecting routes with middleware?                                                  |
| 9. Middleware use cases?                                       | 24. Risk of sensitive logic in Client Components?                                       |
| 10. Fetching data server vs client in App Router?              | 25. Server Actions for form submissions?                                                |
| 11. Vercel vs self-hosted Azure trade-offs?                    | 26. Server Action vs traditional API route?                                             |
| 12. Next.js image optimization?                                | 27. Optimizing Next.js bundle size?                                                     |
| 13. Layouts and nested layout composition?                     | 28. i18n in Next.js at a high level?                                                    |
| 14. loading.js / error.js states?                              | 29. revalidatePath vs revalidateTag?                                                    |
| 15. Streaming SSR and Suspense?                                | 30. Walk me through architecting a full app with static/server pages and RAG API calls. |

ARCHITECTURE — HOW IT FITS TOGETHER

Before LLMs, “AI” mostly meant machine learning, structured broadly as:

Supervised learning — labeled outcomes (e.g., predict house price, classify spam) →

Unsupervised learning — no labels (e.g., group similar customers) →

Reinforcement learning — learning through reward/penalty feedback

Models are built from features, evaluated on held-out data to check they generalize rather than overfit, and tuned using metrics matched to the task (accuracy, precision/recall, RMSE, etc.). This foundation matters for Gen AI because embeddings, evaluation, and grounding all borrow directly from this world — and interviewers often test whether you know the difference between “old AI” and “new AI.”

SEQUENCED CHECKLIST

- ☐ 1. Supervised vs unsupervised vs reinforcement learning
- ☐ 2. Regression vs classification
- ☐ 3. Features, labels, training/validation/test splits
- ☐ 4. Overfitting vs underfitting, generalization
- ☐ 5. Common algorithms conceptually: linear/logistic regression, decision trees, random forests
- ☐ 6. Clustering (unsupervised) — e.g. k-means, conceptually
- ☐ 7. Evaluation metrics: accuracy, precision, recall, F1, RMSE
- ☐ 8. Neural networks basics conceptually
- ☐ 9. What deep learning added over classical ML
- ☐ 10. Responsible AI basics: fairness, bias, transparency, accountability
- ☐ Concept-fluency only — no coding required for your goals here

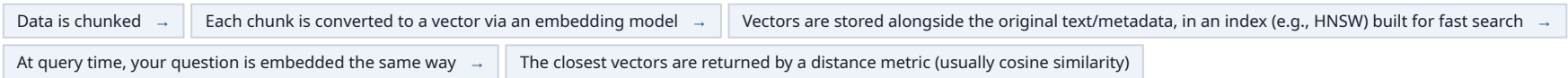
TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1–15                                                          | QUESTIONS 16–30                                                                                                      |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1. Supervised vs unsupervised learning?                                 | 16. Why doesn't more data always mean a better model?                                                                |
| 2. Regression vs classification?                                        | 17. What's data drift, why does it matter in production?                                                             |
| 3. What's overfitting, and how do you prevent it?                       | 18. Classification thresholds and precision/recall trade-offs?                                                       |
| 4. Why split data into train/validation/test?                           | 19. Ensemble methods (random forest, boosting) — why they outperform single models?                                  |
| 5. Precision vs recall — when do you prioritize one?                    | 20. Batch vs online (incremental) learning?                                                                          |
| 6. What's a confusion matrix?                                           | 21. What's regularization, and why is it used?                                                                       |
| 7. How does a decision tree make a prediction, at a high level?         | 22. Handling a highly imbalanced dataset (e.g. fraud detection)?                                                     |
| 8. Classical ML model vs deep learning model?                           | 23. Generative vs discriminative model — link to modern Gen AI?                                                      |
| 9. What's clustering — real-world example?                              | 24. Transfer learning, conceptually?                                                                                 |
| 10. Feature engineering, and why deep learning reduced the need for it? | 25. The six Responsible AI principles (fairness, reliability, privacy, inclusiveness, transparency, accountability)? |
| 11. Bias-variance trade-off?                                            | 26. Explainability vs interpretability?                                                                              |
| 12. Accuracy vs F1 score — why accuracy can mislead?                    | 27. Monitoring a deployed ML model for degrading performance?                                                        |
| 13. Neural network basics — layers, weights, activation?                | 28. A/B testing for validating a model change?                                                                       |
| 14. Reinforcement learning vs supervised learning?                      | 29. Anomaly detection vs classification?                                                                             |
| 15. Parametric vs non-parametric model, conceptually?                   | 30. Explaining a wrong model prediction to a non-technical stakeholder?                                              |

## 9. VECTOR DATABASES

### ARCHITECTURE — HOW IT FITS TOGETHER

A vector database stores embeddings and lets you search by similarity instead of exact match. The pipeline flows as:



This is the retrieval engine that powers RAG — the piece that finds “which of my documents are actually relevant” before handing anything to the LLM.

### SEQUENCED CHECKLIST

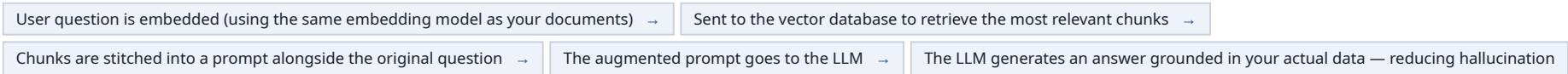
- ☐ 1. What an embedding is (recap from Traditional AI section)
- ☐ 2. Similarity search vs keyword search
- ☐ 3. Distance metrics: cosine similarity, Euclidean distance
- ☐ 4. What an index is (e.g. HNSW) and why brute-force doesn't scale
- ☐ 5. Chunking strategies — size/overlap and retrieval quality
- ☐ 6. Metadata filtering alongside vector search (hybrid search)
- ☐ 7. Popular options conceptually: Pinecone, Azure AI Search, pgvector
- ☐ 8. Hands-on: embed a handful of documents and run a similarity query

### TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1-15                                                                  | QUESTIONS 16-30                                                                     |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1. What is an embedding, and how does it represent meaning numerically?         | 16. What's re-ranking, why add it after retrieval?                                  |
| 2. Semantic (vector) search vs keyword search?                                  | 17. Metadata filtering alongside a vector query?                                    |
| 3. Cosine similarity vs Euclidean distance for text embeddings?                 | 18. Storage/cost trade-offs of vector databases at scale?                           |
| 4. Why can't you brute-force compare against millions of vectors?               | 19. Pinecone vs Azure AI Search vs pgvector, at a high level?                       |
| 5. What's an index like HNSW doing conceptually?                                | 20. Multi-tenant data isolation in a shared vector database?                        |
| 6. Deciding chunk size for a vector database?                                   | 21. Risk of chunking a document badly?                                              |
| 7. What's chunk overlap, why does it matter?                                    | 22. Keeping a vector index in sync with an updating dataset?                        |
| 8. What's hybrid search?                                                        | 23. Sparse vs dense vector representations?                                         |
| 9. What happens if your embedding model changes — do old embeddings still work? | 24. Benchmarking two embedding models for your use case?                            |
| 10. Evaluating whether vector search returns relevant results?                  | 25. Dimensionality reduction and why apply it to embeddings?                        |
| 11. Vector database vs traditional relational database?                         | 26. Designing a system to embed/index millions of documents efficiently?            |
| 12. Updating/deleting a document's embedding when the source changes?           | 27. Latency trade-off: larger accurate index vs smaller faster one?                 |
| 13. Curse of dimensionality and vector search?                                  | 28. Securing a vector database storing sensitive documents?                         |
| 14. Exact vs approximate nearest-neighbor search?                               | 29. Storing embeddings vs on-the-fly embedding at query time?                       |
| 15. Choosing an embedding model for a specific domain?                          | 30. Walk through the full pipeline: raw documents in, relevant chunks out, for RAG. |

ARCHITECTURE — HOW IT FITS TOGETHER

A RAG (Retrieval-Augmented Generation) system connects everything above. The flow is:



Guardrails sit at the input (blocking bad prompts/injection) and output (checking for harmful/ungrounded content) of this pipeline. For more complex tasks, agents extend this pattern further: the LLM can decide to call tools/functions (search, calculators, APIs) across multiple steps, not just generate one answer.

SEQUENCED CHECKLIST

- ☐ 1. LLM basics: tokens, context window, temperature, hallucination
- ☐ 2. Prompt engineering: zero-shot, few-shot, chain-of-thought
- ☐ 3. Recap: embeddings + vector databases as the retrieval engine
- ☐ 4. RAG architecture end-to-end (ingest → chunk → embed → store → retrieve → augment prompt → generate)
- ☐ 5. Guardrails / Responsible AI: content filtering, prompt injection, grounding
- ☐ 6. Fine-tuning vs RAG vs prompt engineering
- ☐ 7. Agents: tool/function calling, multi-step reasoning
- ☐ 8. Azure-specific: Azure OpenAI Service, Azure AI Foundry, Azure AI Search
- ☐ 9. Evaluating a RAG system (relevance, faithfulness, latency, cost)
- ☐ Book and pass AI-901
- ☐ Hands-on: build a mini RAG chatbot (feeds into final integration project)

TOP 30 INTERVIEW QUESTIONS (PRIORITY ORDER)

| QUESTIONS 1–15                                                                   | QUESTIONS 16–30                                                                                                                                                        |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Explain RAG end-to-end.                                                       | 16. What's "groundedness," how do you measure it?                                                                                                                      |
| 2. Why use RAG instead of fine-tuning?                                           | 17. Designing RAG for a large enterprise knowledge base (millions of docs)?                                                                                            |
| 3. What's an embedding, and how does semantic search differ from keyword search? | 18. System prompt vs user prompt?                                                                                                                                      |
| 4. Deciding chunk size for RAG?                                                  | 19. Function calling / tool use in modern LLM APIs?                                                                                                                    |
| 5. Reducing hallucinations in an LLM-based application?                          | 20. LLM agent vs simple RAG chatbot?                                                                                                                                   |
| 6. What's a guardrail — give an example?                                         | 21. Preventing a RAG chatbot from answering out-of-scope questions?                                                                                                    |
| 7. What's prompt injection, how do you defend against it?                        | 22. What's "prompt leaking," how do you mitigate it?                                                                                                                   |
| 8. Chatbot vs "agent"?                                                           | 23. Handling multi-turn conversation context in a RAG chatbot?                                                                                                         |
| 9. Evaluating whether a RAG system gives good answers?                           | 24. Azure OpenAI Service vs calling OpenAI's API directly?                                                                                                             |
| 10. Cost/latency trade-offs of RAG vs a larger context window?                   | 25. What's Azure AI Foundry, how does it fit into building a Gen AI app?                                                                                               |
| 11. Zero-shot vs few-shot vs chain-of-thought prompting?                         | 26. Versioning and testing prompt changes safely in production?                                                                                                        |
| 12. What's "temperature" in an LLM API call?                                     | 27. Cost structure of an LLM API call (input vs output tokens)?                                                                                                        |
| 13. What's a context window, what happens when exceeded?                         | 28. Implementing streaming responses in a chatbot UI?                                                                                                                  |
| 14. Handling a RAG system where retrieval finds irrelevant chunks?               | 29. Vector search re-ranker vs the LLM scoring relevance itself?                                                                                                       |
| 15. Fine-tuning vs RAG — a scenario for each?                                    | 30. Walk me through architecting a production RAG chatbot end-to-end, including how it's containerized/deployed (Docker + Kubernetes/App Service + Terraform + Azure). |

PART 3 — SENIOR-LEVEL EXTRAS

Ongoing throughout — don't skip.

SYSTEM DESIGN (2-3×/WEEK, 45 MIN)

- ☐ Practice the framework: requirements → back-of-envelope estimation → high-level design → deep dive → trade-offs
- ☐ Classics: URL shortener, rate limiter, notification system
- ☐ Your-stack-specific: “design a scalable RAG-based support chatbot for X users on Azure”

BEHAVIORAL (STAR FORMAT)

- ☐ Story 1: a production incident you resolved
- ☐ Story 2: a technical disagreement and how it was resolved
- ☐ Story 3: mentoring someone
- ☐ Story 4: a project that failed and what you learned
- ☐ Story 5: influencing without authority
- ☐ Story 6: a scaling/performance challenge you solved

QUICK REFERENCE — CERT ORDER

CERT ORDER

- 1. AZ-900 (Azure Fundamentals)
- 2. Terraform Associate (004)
- 3. AI-901 (Azure AI Fundamentals)

JOB-HUNT REMINDER

- Keep applications running every single week
- Roadmap is supporting evidence — not the whole job hunt
- Do not pause applying for certifications